

Revised Design: Neuroevolution and Genetic Algorithms

A complete engineering baseline that integrates all eleven fixes from the independent Critical Review into Design Document v1.0. This document supersedes v1.0 and can be read standalone.

Item	Value
Document version	2.0 — Revised (integrates Critical Review fixes C1–C11)
Project type	Deep Learning / AI / Simulation
Primary language	Python
Simulation engine	PyGame
Neural framework	PyTorch — batched population inference (Section 11.3)
Evolution	Real-coded GA on a fixed-topology MLP (core, Phases 1–6) + fully specified NEAT-style topology extension (Phase 7, Appendix C)
Reproducibility	Every run tagged with a CPU-deterministic or GPU tier (Section 15.4)
Target platform	Desktop; optional web dashboard
Status	Design baseline v2.0

Change Log — v1.0 → v2.0

Each row maps one fix from the Critical Review to where it now lives in this document. Fixes are also flagged inline with **[NEW]** or **[REVISED]** tags on the relevant headings.

Fix	Where it landed	Summary
C1	5.1, 14.2, Phase 7, Appendix C	Scope now states a fixed-topology core plus a fully specified NEAT extension
C2	12.2, Experiment E	Default crossover changed to blend/arithmetic; competing-conventions risk documented; mutation-only ES added as a comparison arm
C3	11.3, FR-04, NFR-Performance	Batched population inference is now a core architectural requirement, prototyped in Phase 1.5
C4	13.0–13.1, Experiment D	Fitness components calibrated and normalized before weighting; weighting itself is now an experiment
C5	15.4, 17.1	Explicit determinism checklist; CPU/GPU results tagged as separate reproducibility tiers
C6	16.1–16.2	MVP and Advanced-phase dashboard mockups separated so neither misrepresents the other's scope

Fix	Where it landed	Summary
C7	18.6	Compute budget is estimated from a Phase 1.5 benchmark before any full sweep is scheduled
C8	2	Related Work section grounds the design in prior neuroevolution / artificial-life research
C9	19.4	Hall-of-fame evaluation added so co-evolutionary (predator/prey) progress can be measured absolutely
C10	10.5	Explicit per-step energy-balance equation added
C11	19.5, Appendix D	Statistical comparison protocol specified: Mann-Whitney U, bootstrap CI, Holm-Bonferroni correction

1. Executive Summary

The proposed system is a 2D artificial-life simulation in which autonomous organisms learn and evolve behaviors through neural-network controllers optimized by a genetic algorithm. The core evolutionary loop, environment design, and staged development philosophy are unchanged from v1.0 — this revision does not redesign the project, it closes eleven specific gaps between what v1.0 claimed and what it actually specified.

Two scope decisions anchor this revision. First, the system is now described accurately at every phase: Phases 1–6 implement fixed-topology neuroevolution — a genetic algorithm optimizing the weights of a fixed 12-32-16-3 network — and Phase 7 implements a fully specified NEAT-style topology extension (innovation tracking, compatibility-distance speciation, and complexification; see Appendix C), rather than an under-specified “NEAT-style” label applied to the whole system from the start. Second, population-scale neural inference is batched from the first neural-network prototype (Section 11.3), not deferred to a late-stage performance fix, because batching is what makes the real-time performance target achievable at all.

Core Research Question

Can useful adaptive behavior emerge in autonomous agents when neural-network controllers are optimized by evolutionary selection rather than directly programmed? This revision adds a secondary question: does that behavior depend on how fitness is weighted (Experiment D), and does it survive the transition from a fixed topology to full topology evolution (Phase 7)?

2. Related Work [NEW]

This system sits within an established line of work on evolving behavior rather than programming it directly. Sims demonstrated that both morphology and control could be evolved together for creatures competing over a shared resource, establishing the basic evolved-behavior-through-competition template this project's predator/prey phase follows. Stanley and Miikkulainen's NEAT introduced historical markings and compatibility-based speciation specifically to solve the problem this document calls out in C2 and C1 — meaningful crossover between differently-structured genomes — and Appendix C follows that formulation directly rather than inventing a new one. Prior artificial-life platforms built around an environment/agent/genome loop (e.g., Polyworld, Framsticks) establish precedent for the layered architecture used here. Positioning this project against that lineage clarifies which design choices are precedented (tournament selection, speciation via compatibility distance) and which are this project's own experimental questions (fitness weighting, crossover-method comparison).

3. Problem Definition

Traditional game and simulation agents are often controlled by explicit if/then rules. Such rules are effective for deterministic behavior, but they provide no mechanism for discovering new strategies. This project addresses that limitation by making the decision policy learned rather than authored: a neural network defines behavior, and evolution searches for the parameters (and, in Phase 7, the structure) that improve survival and task performance.

Rule-based approach	Evolutionary approach
IF food is near → move toward food IF predator is near → move away	Sensors → Neural Network → Actions → Fitness → Selection → Mutation → New generation

4. Objectives and Success Criteria [REVISED]

Objective	Success criterion
Environment	Stable real-time 2D world with deterministic and randomized configurations
Autonomous agent	Agent acts from sensor observations without a hard-coded task policy
Neural controller	Controller maps normalized observations to movement/action outputs, run as a single batched forward pass across the population
Evolution	Fitness-driven selection, crossover, mutation, and generation replacement work reproducibly, with crossover method itself compared, not assumed
Learning evidence	Median fitness improves across generations under controlled seeds, measured against normalized fitness components
Generalization	Evolved agents can be evaluated on unseen environment layouts
Inference performance	Batched forward pass sustains the configured real-time FPS at the baseline population (250), benchmarked at the Phase 1.5 exit gate
Reproducibility tiering	Every experiment is tagged CPU-deterministic or GPU; results from different tiers are never pooled as equivalent
Statistical rigor	Condition comparisons are reported with a significance test, an effect size or confidence interval, and a multiple-comparison correction
Scope honesty	The system description matches the implemented algorithm at every phase; no under-specified NEAT claim before Phase 7
Analytics	Dashboard reports fitness, survival, resource use, diversity, and runtime statistics, scoped correctly to MVP vs. advanced phase
Engineering quality	Automated tests cover core modules, including batched-vs-per-agent inference equivalence, and experiments are reproducible from configuration files

5. Scope and Constraints [REVISED]

5.1 In Scope

- 2D continuous simulation space.
- Autonomous organisms with limited local perception.

- Food/resource seeking and obstacle avoidance.
- Energy, health, age, movement cost, and death, with an explicit energy-balance equation (Section 10.5).
- Genetic representation of fixed-topology neural-network parameters (Phases 1–6).
- Selection, elitism, crossover (compared across three methods), mutation, and generation management.
- Population-level batched neural inference from the first working prototype (Phase 1.5 onward).
- A fully specified NEAT-style topology-evolution extension — innovation tracking, compatibility-distance speciation, complexification — scoped to Phase 7 (Appendix C).
- Experiment logging, statistical comparison, and reproducibility-tier tagging.
- Optional predator/prey extension with hall-of-fame evaluation for non-stationary co-evolution.

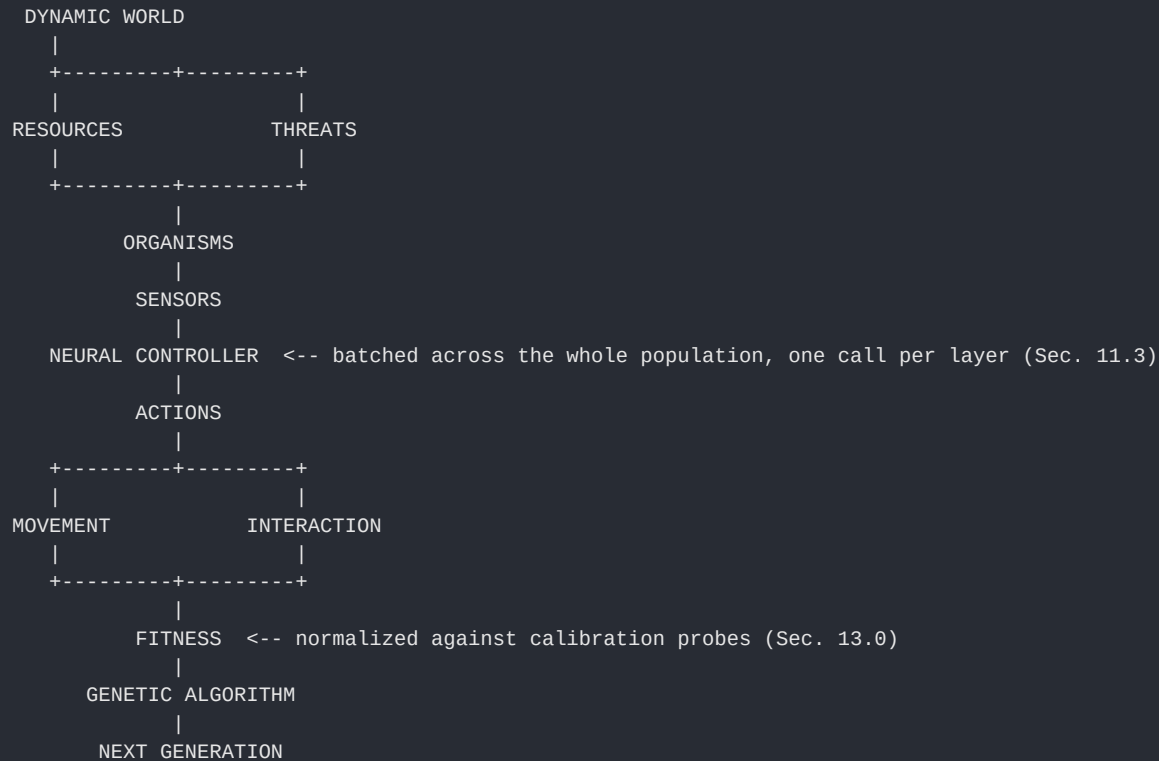
5.2 Out of Scope for the First Release

- Photorealistic 3D environments.
- Real-world robot hardware integration.
- Large-scale distributed evolution across multiple machines.
- Biologically accurate genetics or ecology.
- Medical or safety-critical decision making.
- Topology evolution before Phase 7 — Phases 1–6 use the fixed 12-32-16-3 architecture throughout.

5.3 Constraints

Constraint	Design response
Compute budget	Begin with 100–250 agents and a lightweight, batched dense network
Real-time rendering	Separate simulation update rate from rendering frame rate; headless mode for scheduled experiments (Section 15.2)
Reproducibility	Explicit random seeds, versioned configs, and a CPU/GPU reproducibility tier on every experiment (Section 15.4)
Complexity control	Staged releases from MVP to advanced NEAT ecosystem, with batching and statistics infrastructure pulled forward to Phase 1.5/4.5
Experimental compute budget	Benchmark steps/sec in Phase 1.5; size the full experiment sweep to the measured budget (Section 18.6), not the other way around

6. System Concept



The system remains a closed evolutionary loop: agents perceive local state, the neural controller converts observations into actions, the simulation evaluates consequences, and the genetic algorithm uses fitness to create the next population. The two annotations above are the structural changes from v1.0: the neural controller step is a population-wide batched operation, not a per-agent one, and fitness is computed from calibrated, normalized components rather than raw counts at hand-picked coefficients.

7. Functional Requirements [REVISED]

ID	Requirement
FR-01	Initialize environment with configurable dimensions, resource density, obstacles, and population size
FR-02	Spawn organisms with valid genomes and deterministic identifiers
FR-03	Calculate sensor observations from nearby entities and obstacles
FR-04	Run the neural controller as a single batched forward pass across the live population and generate continuous per-agent action outputs (revised — was per-agent)
FR-05	Apply physics, movement costs, collisions, eating, damage, and death, using the explicit energy-balance equation in Section 10.5
FR-06	Accumulate per-agent fitness during each generation from normalized components
FR-07	End a generation when a configured time limit or terminal condition is reached
FR-08	Select parents using a configurable selection strategy
FR-09	Generate offspring through a configurable crossover method (blend, uniform, or none) and mutation

ID	Requirement
FR-10	Log generation-level and agent-level metrics, tagged with a reproducibility tier
FR-11	Pause, resume, accelerate, reset, and save the simulation
FR-12	Run predefined experiments from configuration files in headless mode, without UI interaction
FR-13	Tag every experiment run with a reproducibility tier (CPU-deterministic / GPU) and the full seed set (torch/numpy/random) in metadata
FR-14	Compute and log a statistical comparison (test statistic, confidence interval, corrected p-value) between conditions within an experiment

8. Non-Functional Requirements [REVISED]

Category	Requirement
Performance	The batched population inference path (Section 11.3) must sustain the configured real-time FPS for the baseline population on reference hardware; this is measured and confirmed at the Phase 1.5 exit gate, not discovered later
Reproducibility	Same seed set + same configuration + same reproducibility tier produces equivalent results; CPU and GPU tiers are never compared as if bit-identical (Section 15.4)
Maintainability	Separate environment, agent, neural, evolution, visualization, and analytics modules
Extensibility	New sensors, actions, organisms, or evolutionary strategies should be addable without rewriting the engine or the batched-inference path
Observability	Runtime and evolution statistics, including significance test results, must be accessible during and after experiments
Robustness	Invalid values, NaN weights, malformed genomes, and empty populations must be handled safely
Usability	Controls and statistics remain understandable during live demonstrations; the MVP dashboard shows only metrics that exist at that phase (Section 16.1)

9. Architecture [REVISED]

```

+-----+
| Presentation Layer | PyGame renderer / Dashboard (MVP + Advanced mockups)
+-----+
|
+-----v-----+
| Simulation Layer   | engine / physics / collisions / headless-or-GUI mode
+-----+
|
+-----v-----+
| Agent Layer       | organisms / sensors / actions / energy balance
+-----+
|
+-----v-----+
| AI Layer          | batched neural inference / genome
+-----+
|
+-----v-----+
| Evolution Layer    | selection / crossover (3 methods) / mutation
+-----+
|
+-----v-----+
| Analytics Layer (NEW) | statistical comparison / significance testing
+-----+
|
+-----v-----+
| Persistence Layer  | configs / metrics / checkpoints / reproducibility tags
+-----+

```

9.1 Module Responsibilities

Module	Responsibilities
simulation.engine	Main loop, time steps, generation lifecycle, GUI/headless mode orchestration
simulation.environment	World state, resources, obstacles, spawn/reset logic
simulation.physics	Position, velocity, steering, fixed-timestep energy consumption, collision responses
agents.organism	Agent state, genome binding, actions, lifecycle, energy balance (Section 10.5)
agents.sensors	Ray casting, target detection, normalized observations
neural.network	Network architecture definition and weight (de)serialization
neural.batched_inference	Population-level batched forward pass across all live agents (Section 11.3)
neural.genome	Genome vector representation, packing/unpacking
evolution.genetic_algorithm	Population evolution and generation creation
evolution.selection	Tournament, roulette, rank, and elitism strategies
evolution.crossover	Blend/arithmetic (default), uniform, and mutation-only conditions
evolution.mutation	Gaussian mutation, clipping, adaptive mutation extension
evolution.neat	Innovation tracking, compatibility distance, speciation, complexification (Phase 7, Appendix C)

Module	Responsibilities
analytics.statistics	Mann-Whitney U, bootstrap CI, Holm-Bonferroni correction (Section 19.5, Appendix D)
visualization.renderer	World rendering and entity visualization
visualization.dashboard	Metrics, graphs, controls, best-agent inspector; MVP and Advanced variants
analytics.experiments	Experiment logging, aggregation, plots, and summary reports

10. Environment Design

10.1 World Model

```
World(width, height)
- resources: list[Food]
- obstacles: list[Obstacle]
- agents: list[Organism]
- time: float
- generation: int
- random_seed: int
- reproducibility_tier: "cpu-deterministic" | "gpu" # NEW, fixes C5
```

10.2 Food

Property	Description
position	2D coordinates
energy_value	Energy restored when consumed
respawn	Optional probability or timed respawn
radius	Collision/eating radius

10.3 Obstacles and World Boundaries

Obstacles are static geometric objects in the MVP, displayed distinctly by the renderer and treated as detectable hazards by the sensor subsystem; the advanced version may introduce moving obstacles or changing terrain. Agents are not allowed to leave the world. Boundary contact clamps position and applies a small collision penalty, which remains the recommended baseline because it avoids the instability of velocity reflection.

11. Agent and Sensor Design [REVISED]

11.1 Agent State

Field	Type	Purpose
id	int	Unique identifier
position	Vector2	Current location

Field	Type	Purpose
velocity	Vector2	Movement state
heading	float	Orientation
energy	float	Survival resource (Section 10.5)
health	float	Damage state
age	int/float	Lifetime
fitness	float	Evolution score (normalized, Section 13)
genome	array	Neural parameters
alive	bool	Lifecycle state

11.2 Ray-Based Vision

```

    ray -90 deg
      \
ray -60 deg ----- ray +60 deg
      AGENT
ray -30 deg ----- ray +30 deg
      /
    ray +90 deg

```

Seven rays are recommended in the baseline configuration: -90, -60, -30, 0, +30, +60, +90 degrees. Each ray returns normalized features describing the nearest relevant object within a maximum range.

11.3 Sensor Encoding and Action Space

Observation	Example encoding
Food distance	[0,1], where 0 is near and 1 is beyond max range
Food bearing	[-1,1], normalized left/right direction
Threat distance	[0,1]
Threat bearing	[-1,1]
Obstacle distance	[0,1]
Energy	[0,1]
Speed	[0,1]

The observation vector must remain fixed within an experiment so genome dimensions stay compatible. Any change to input ordering is a new model configuration and experiment version.

```

steering  = tanh(output[0])    # -1 left to +1 right
accel     = sigmoid(output[1]) # 0 to 1
eat_signal = sigmoid(output[2]) # threshold for eating

```

11.4 Energy Balance **[NEW]**

v1.0 referenced “movement costs” and made survival depend on energy, but never wrote down the actual state transition. This closes that gap (fixes C10):

```
energy[t+1] = energy[t]
              - metabolic_base_cost
              - k_accel * accel_output**2
              - k_steer * abs(steering_output)
              + eaten_energy_value * eat_event

eat_event = 1 if (eat_signal > eat_threshold and within_eating_radius(agent, food)) else 0
death when energy <= 0 or health <= 0
```

Symbol	Meaning	Recommended baseline
metabolic_base_cost	Passive per-step energy cost	0.01 (config)
k_accel	Cost coefficient for acceleration	0.05 (config)
k_steer	Cost coefficient for steering magnitude	0.02 (config)
eat_threshold	Sigmoid threshold that triggers an eat attempt	0.5 (config)

12. Neural Network Design [REVISED]

The neural controller is a feed-forward multilayer perceptron for the fixed-topology system (Phases 1–6). It is intentionally small so hundreds of agents can be simulated concurrently, and — as of this revision — is always evaluated as a batched population, not per agent (Section 12.3).

```
Input: 12-24 normalized observations
|
Dense 32 | ReLU
Dense 16 | ReLU
Output 3
|
steering / accel / eat
```

Parameter	Baseline
Input size	12–24 depending on sensor configuration
Hidden 1 / Hidden 2	32 neurons / 16 neurons
Activation	ReLU
Output	3 (steering: tanh; accel, eat: sigmoid)

12.1 Genome Encoding

All weights and biases are flattened into a one-dimensional vector. The neural-network class provides deterministic packing/unpacking so the same genome always reconstructs the same controller.

```
Genome = [w1..., b1..., w2..., b2..., w3..., b3...]
```

12.2 Why a Fixed Network First

A fixed topology isolates the evolutionary problem to parameter optimization. Phase 7 adds topology evolution as a distinct, fully-specified experimental condition (Appendix C) rather than mixing multiple sources of complexity from the beginning — this is the resolution to C1: the scope is now honest about which phases actually do NEAT-style evolution.

12.3 Batched Population Inference [NEW]

All agents share the same fixed topology but not the same weights, so a single shared `nn.Module` cannot be reused across the population — batching has to happen at the weight-matrix level. Each layer's weights are stacked across the population into an $(N, \text{in}, \text{out})$ tensor, and one batched matrix multiply replaces N separate forward calls (fixes C3):

```
observations = stack([agent.sensors.observe(world) for agent in population]) # (N, input_dim)

# Per layer: weights stacked as (N, in, out), biases as (N, out)
h1 = relu(bmm(observations.unsqueeze(1), W1) + b1) # one batched call, not N calls
h2 = relu(bmm(h1, W2) + b2)
out = bmm(h2, W3) + b3
actions = stack([tanh(out[:, 0]), sigmoid(out[:, 1]), sigmoid(out[:, 2])], dim=1)

apply_actions(population, actions) # single population-wide step, once per simulation tick
```

Design Rule

Population-level batching is a Phase 1.5 exit criterion, benchmarked before Phase 2 begins — not a Phase 8 packaging concern. This turns roughly 250 Python-level PyTorch calls per simulation step into about 3 (one batched call per layer), which is the difference between hitting and missing the real-time NFR in Section 8.

13. Genetic Algorithm Design [REVISED]

```
for generation in range(max_generations):
    simulate_population() # batched inference, Section 12.3
    evaluate_fitness() # normalized components, Section 14
    elites = select_elites(population)
    parents = select_parents(population)
    children = crossover_and_mutate(parents) # method is a config choice, Section 13.2
    population = elites + children
    log_metrics() # tagged with reproducibility tier, Section 15.4
```

13.1 Selection

Tournament selection remains the recommendation: simple, configurable, and less sensitive to extreme fitness scales than raw roulette-wheel selection. A small elite set is preserved unchanged to prevent loss of the best solution found so far.

13.2 Crossover [REVISED]

Method	Description	Role in v2.0
Blend / Arithmetic (default)	Weighted average of numeric genes	Default baseline — gentler on real-valued weight vectors than a positional swap
Uniform	Each gene is chosen independently from either parent	Comparison condition (Experiment E)
Mutation-only (no crossover)	Selection + mutation only, no recombination	Comparison baseline: tests whether crossover helps at all for a fixed-topology weight vector

Design Rule — fixes C2

Positionally swapping or averaging weights between two independently-good parents at the same index can produce a child worse than either parent, because two networks can compute equivalent functions with their hidden units in a different internal order (a permutation symmetry) — the “competing conventions” problem. This is the same problem NEAT’s historical markings solve structurally (Appendix C); until Phase 7, this design mitigates it empirically by comparing crossover methods (Experiment E) rather than assuming uniform crossover is best by default.

13.3 Mutation

```
for gene in genome:
    if random() < mutation_rate:
        gene += Normal(0, mutation_strength)
        gene = clip(gene, min_weight, max_weight)
```

Mutation prevents premature convergence and introduces new candidate behaviors. Mutation rate and strength remain configuration parameters and key experimental variables (Experiment A).

14. Fitness and Reward Design [REVISED]

Fitness defines what the evolutionary system considers successful. v1.0 combined raw counts at hand-picked coefficients; v2.0 calibrates each component’s natural scale first, then combines normalized components, which removes the scale-domination risk called out as C4.

14.1 Calibration [NEW]

Before coefficients are treated as fixed, run calibration episodes with (a) a random policy and (b) a stationary (“do nothing”) policy, and record each raw component’s mean value as its reference scale.

```
# One-time calibration pass, run in Phase 3 before any coefficient is locked:
reference_scale[term] = mean(raw_component[term] over calibration episodes)

# Per-agent fitness at generation end:
component[term] = raw_component[term] / max(reference_scale[term], epsilon)
fitness = sum(weight[term] * component[term] for term in fitness_terms)
```

14.2 Fitness Terms

Term	What it rewards	v2.0 baseline weight (normalized)
survival_time	Staying alive	1.0
food_consumed	Resource acquisition	1.0
distance_explored	Exploration, capped to avoid reward hacking	1.0
successful_reproduction	Producing viable offspring (advanced phase)	1.0
collisions	Penalized	-1.0

Equal nominal weights on normalized components are the v2.0 default; whether a different weighting produces better or more interesting behavior is no longer an assumption — it's Experiment D (Section 18.4).

14.3 Fitness Failure Modes

Failure mode	Example	Mitigation
Reward hacking	Agent spins to maximize movement score	Cap exploration contribution; require meaningful displacement
Stalling	Agent stays still to avoid collision	Energy drains with time (Section 10.5); reward for resource collection
Safe corner	Agent camps beside food indefinitely	Randomize food positions; periodic resource relocation
Over-aggression	Predators waste energy chasing unreachable targets	Include energy and capture efficiency in fitness

15. Reproduction and Species [REVISED]

15.1 Reproduction

The MVP uses generation-level replacement without physical reproduction in the environment. The advanced version adds explicit reproduction where healthy, mature agents produce offspring, separating genetic inheritance from population-management so both can be studied independently.

```
eligible = age >= MIN_AGE and energy >= MIN_ENERGY and fitness >= MIN_FITNESS
child = crossover(parent_a.genome, parent_b.genome) # method per Section 13.2
child = mutate(child)
```

15.2 Species and NEAT-Style Topology Evolution [REVISED]

Species grouping and topology evolution are implemented together as the Phase 7 extension, fully specified in Appendix C: innovation tracking, compatibility-distance-based speciation, fitness sharing, and complexification from a minimal starting topology. This section intentionally defers detail to the appendix rather than repeating it inline — the resolution to C1 is that the system is honestly scoped as fixed-topology through Phase 6, and Phase 7 is a real, fully-specified NEAT implementation, not a one-paragraph stretch goal.

16. Simulation Engine and Runtime Loop [REVISED]

```
INITIALIZATION
  load configuration
  set random seed (torch + numpy + random)
  tag reproducibility tier (cpu-deterministic | gpu)
  create world
  create population

EACH FRAME
  read controls
  update sensors
  run batched neural network forward pass across live population <- Sec 12.3
  apply actions
  update physics (fixed timestep + accumulator)
  resolve interactions
  update energy / health / age <- Sec 10.5
  accumulate fitness (normalized components) <- Sec 14
  render (skipped entirely in headless mode)

GENERATION END
  finalize fitness
  log statistics (tagged with reproducibility tier)
  select parents
  crossover (method per config: blend | uniform | none) <- Sec 13.2
  mutate
  create next generation
  reset world or randomize according to experiment
```

16.1 Execution Modes [NEW]

Two explicit entry points replace the implicit “fast-forward while maybe rendering” behavior of v1.0: an interactive **GUI mode** for live demonstration, and a **headless mode** with no rendering at all for scheduled experiments, which is what actually delivers the “up to 20× fast-forward” figure in Appendix A.

```
python -m app.run --config configs/baseline.json --mode gui
python -m experiments.run --config configs/mutation_rate.json --seeds 10 --mode headless --device cpu
```

16.2 Time Management

Mechanism	Purpose
Simulation timestep	Stable, frame-rate-independent physics via a fixed-timestep-with-accumulator pattern
Render FPS	Controls visualization frequency; irrelevant in headless mode
Fast-forward factor	Accelerated evolution during headless experiments, up to 20x
Generation duration	Bounds computational cost per generation

16.3 Determinism [REVISED]

- Fix seeds for torch, numpy, and random at experiment start; record all three in `metadata.json`, not a single combined “seed” field.

- Call `torch.use_deterministic_algorithms(True)` and pin `torch.set_num_threads()` to a fixed value for CPU runs.
- Tag every experiment's metadata with `device: cpu|gpu` and treat CPU and GPU runs as separate reproducibility tiers — never pool or directly compare them as bit-identical.

17. UI/UX and Analytics Dashboard [REVISED]

17.1 MVP Dashboard (Phases 1–5, no species metric) [NEW]

```
+-----+
| GENERATION 384 | POPULATION 250 |
+-----+
|                               | STATISTICS | |
| LIVE WORLD                | Best fitness |
|                               | Avg / Median fitness |
| food  obstacle            | Alive      | ### |
| agent >                   | Food consumed |
|                               | Avg lifespan |
+-----+
| Fitness history: /----- |
| Controls: [Pause] [x1] [x10] [Next Gen] [Save] [Reset] |
+-----+
```

17.2 Advanced-Phase Dashboard (Phase 6+, species and co-evolution) [NEW]

```
+-----+
| GENERATION 384 | POPULATION 250 | SPECIES 6 |
+-----+
|                               | STATISTICS |
| LIVE WORLD (predator/prey) | Best / Avg fitness |
|                               | Species count |
| food  obstacle  predator  | Hall-of-fame delta |
| prey >                   | Food consumed / lifespan|
+-----+
| Fitness history (per species): /----- |
| Controls: [Pause] [x1] [x10] [Next Gen] [Save] [Reset] |
+-----+
```

Separating these two mockups resolves C6: the MVP build is never asked to show a species count that doesn't exist yet, and the advanced build's co-evolution-specific panels (hall-of-fame delta, per-species fitness) are documented where they actually apply.

17.3 Best-Agent Inspector

- Current fitness, age, energy, health, and movement speed.
- Sensor rays and current observations.
- Neural-network graph with active outputs.
- Genome identifier and parent lineage.
- Behavior timeline for the current generation.

18. Data and Experiment Management [REVISED]

```
experiments/  
  EXP-001/  
    config.json  
    metadata.json          # now includes reproducibility_tier, torch/numpy/random seeds, device  
    generation_metrics.csv  
    agent_metrics.csv  
    stats_summary.json     # NEW: test statistic, CI, corrected p-value per comparison  
    best_genome.json  
    checkpoints/  
  EXP-002/  
  ...
```

18.1 Generation Metrics

Field	Description
generation	Generation index
best_fitness / mean_fitness / median_fitness / std_fitness	Population fitness distribution
alive_count	Agents alive at generation end
food_consumed	Total resource consumption
avg_lifespan	Mean lifespan
species_count	Number of identified species (Phase 7 onward)

18.2 Configuration Example [REVISED]

```
{  
  "population_size": 250,  
  "generation_steps": 5000,  
  "mutation_rate": 0.05,  
  "mutation_strength": 0.10,  
  "elite_fraction": 0.10,  
  "crossover_method": "blend",  
  "fitness_weights": {"survival": 1.0, "food": 1.0, "exploration": 1.0, "collisions": -1.0},  
  "sensor_count": 7,  
  "food_count": 80,  
  "obstacle_count": 12,  
  "seed": {"torch": 42, "numpy": 42, "random": 42},  
  "reproducibility_tier": "cpu-deterministic",  
  "device": "cpu"  
}
```

19. Experimental Methodology [REVISED]

Each experiment changes one primary variable while keeping the rest of the configuration fixed. Multiple random seeds are required for conclusions that are robust to initialization noise. Experiments D and E are new in v2.0; A–C are unchanged from v1.0 except that network-capacity conditions are relabeled NC1–NC3 to avoid clashing with the

fix IDs used in the change log.

19.1 Experiment A — Mutation Rate

Condition	Mutation rate
A1	0.01
A2	0.05
A3	0.10
A4	0.20

19.2 Experiment B — Population Size

Condition	Population
B1	50
B2	100
B3	250
B4	500

19.3 Experiment NC — Network Capacity

Condition	Architecture
NC1	12 -> 8 -> 3
NC2	12 -> 16 -> 8 -> 3
NC3	12 -> 32 -> 16 -> 3

19.4 Experiment D — Fitness Weighting [NEW]

Condition	Weighting emphasis
D1	Baseline: equal normalized weights (Section 14.2 default)
D2	Resource-weighted: food_consumed weight doubled, survival_time halved
D3	Exploration-weighted: distance_explored weight raised, survival_time lowered
D4	Survival-weighted: survival_time weight raised, matching v1.0's original skew, for comparison

19.5 Experiment E — Crossover Method [NEW]

Condition	Method
E1	Blend / arithmetic (v2.0 default)
E2	Uniform (v1.0 default)
E3	Mutation-only, no crossover

19.6 Compute Budget Estimate [NEW]

Benchmark steps/second once during Phase 1.5 at the baseline population size, then extrapolate before committing to a full sweep (fixes C7):

```
est_wallclock = (conditions * seeds * generations * generation_steps * population) / measured_steps_per_second

# A(4) + B(4) + NC(3) + D(4) + E(3) = 18 conditions
# 18 conditions x 10 confirmatory seeds = 180 runs minimum

# Recommended two-stage strategy:
#   Stage 1 (exploratory): 3 seeds/condition, identify promising conditions
#   Stage 2 (confirmatory): 10 seeds/condition, on the conditions worth reporting
```

20. Evaluation Plan [REVISED]

20.1 Primary Metrics

Metric	Purpose
Best / Mean / Median fitness	Strongest strategy found / population-wide improvement / outlier-robust central tendency
Convergence generation	Speed of evolution
Survival time / Food efficiency	Behavioral effectiveness / resource use per unit energy
Generalization score	Performance on unseen worlds

20.2 Baselines

At least one non-evolutionary baseline is included: a deterministic hand-coded controller as a reference point for basic task performance, and a randomly initialized, untrained neural population as a no-learning baseline.

20.3 Generalization Test

```
TRAINING: evolve on layouts A1, A2, A3
|
FREEZE BEST CONTROLLERS
|
TEST: evaluate on unseen layouts B1, B2, B3
|
compare train vs. unseen performance
```

20.4 Co-evolutionary Evaluation — Hall of Fame [NEW]

Section 19's framing ("median fitness improves across generations") assumes a stationary fitness landscape. Once Phase 6 introduces predator/prey, that assumption breaks: a prey's fitness depends on the current predator population's skill and vice versa, so within-generation fitness improving can mean only "keeping pace" with a co-evolving opponent (Red Queen dynamics), and can legitimately oscillate rather than climb. Fixes C9 by evaluating each generation's best controllers against a frozen archive of historical opponents (best-of-generation snapshots

taken every N generations), in addition to same-generation relative fitness, so absolute progress remains measurable.

20.5 Statistical Comparison Protocol [NEW]

For each experiment, compare conditions pairwise using the Mann-Whitney U test on final fitness across the 10 seeds, since normality shouldn't be assumed at that sample size; report a bootstrap confidence interval on the median alongside the test statistic; and apply a Holm-Bonferroni correction across all pairwise comparisons run within an experiment before calling any difference significant. Full procedure in Appendix D.

21. Development Plan [REVISED]

Phase	Main work	Exit criterion
Phase 1	Environment + renderer + basic physics	Agents move and collide reliably
Phase 1.5 (NEW)	Prototype batched population inference (Sec. 12.3); benchmark steps/sec	Measured steps/sec at 250 agents feeds the Sec. 19.6 compute budget
Phase 2	Sensors + fixed neural controller, batched from the start	Agents react to observations through batched NN outputs
Phase 3	Fitness calibration/normalization (Sec. 14.1) + genetic algorithm + crossover comparison arm	Fitness improves over controlled runs; crossover variants compared
Phase 4	Analytics + checkpoints + determinism checklist (Sec. 16.3)	Experiments are logged, repeatable, and tagged by reproducibility tier
Phase 4.5 (NEW)	Related-work write-up (Sec. 2) + statistical protocol implementation (Sec. 20.5)	Comparison pipeline is ready before any headline experiment runs
Phase 5	Generalization experiments + Experiment D (fitness weighting)	Unseen layouts and fitness-weighting sensitivity both evaluated
Phase 6	Predator/prey + reproduction + hall-of-fame evaluation (Sec. 20.4)	Multi-agent ecosystem is stable; co-evolution progress measured against frozen checkpoints
Phase 7	NEAT-style extension per Appendix C: innovation tracking, speciation, complexification	Topology evolution works on an isolated benchmark, fully matching the Appendix C specification
Phase 8	Packaging + final report; dashboard mockups already reconciled with actual scope (Sec. 17)	Demo, documentation, tests, and final experiment results complete

22. Testing Strategy [REVISED]

22.1 Unit Tests

- Sensor ray intersection and normalization.
- Genome serialization/deserialization.
- Neural-network output dimensions and value ranges.
- **Batched forward pass matches per-agent forward pass** within floating-point tolerance, for a random sample of genomes (fixes C3 correctness).

- **Fitness normalization:** calibration reference scale computed correctly against known synthetic raw components (fixes C4).
- Mutation bounds and mutation probability.
- Crossover output length and valid parameter ranges, for all three crossover methods.
- Fitness calculation for known synthetic cases.
- Collision handling and energy updates against the Section 10.5 equation.

22.2 Integration Tests

- Run one complete generation with a tiny population.
- Checkpoint and restore a genome without changing outputs.
- Reset the environment and verify entity counts.
- Run a seeded experiment twice and compare metrics.
- **Same-seed run on the CPU-deterministic tier reproduces identical generation metrics across two runs** (fixes C5).

22.3 Performance Tests

- Measure steps per second at population sizes 50, 100, 250, and 500.
- **Measure batched vs. per-agent inference throughput** at the same population sizes, to confirm the batching speedup (fixes C3).
- Measure rendering overhead separately from simulation overhead.
- Profile neural inference and collision detection.

23. Risk Analysis [REVISED]

Risk	Impact	Likelihood	Mitigation / status in v2.0
Fitness hacking	High	Medium	Bounded, normalized reward terms and behavioral diagnostics (Sec. 14)
Premature convergence	High	Medium	Mutation, elite limits, diversity tracking, optional species
Simulation too slow	High	Medium	Addressed by design: batched inference is a Phase 1.5 exit criterion, not a late-stage fix (Sec. 12.3)
Non-reproducible results	High	Medium	Addressed by design: seeds, configs, versioning, and the determinism checklist (Sec. 16.3)
Feature creep	Medium	High	Freeze MVP before adding predators/species/NEAT
Visualization distracts from research	Medium	Medium	Keep analytics separate from core simulation logic
Underpowered / uncorrected statistics	Medium	Medium (new)	Addressed by design: Mann-Whitney U + bootstrap CI + Holm-Bonferroni (Sec. 20.5)

24. Security and Stability

- Validate configuration values before starting an experiment.
- Reject NaN/Inf values in genomes, fitness values, and physics state.
- Cap population size, generation duration, and checkpoint frequency.
- Use safe serialization formats: JSON for metadata, NumPy-safe arrays for numeric data.
- Keep the simulation sandboxed; evolving agents do not execute arbitrary code.
- Record software version and configuration hash for every experiment.
- **Validate that the recorded reproducibility tier matches the actual runtime device** before accepting an experiment as confirmatory (new).

25. Deployment and Packaging [REVISED]

25.1 Desktop Application

```
python -m app.run --config configs/baseline.json --mode gui
python -m experiments.run --config configs/mutation_rate.json --seeds 10 --mode headless --device cpu
```

25.2 Optional Web Dashboard

A FastAPI backend can expose experiment status and stored metrics, while a React frontend visualizes generations, fitness curves, species counts, hall-of-fame deltas, and best-agent statistics. This remains an optional presentation layer, not coupled to the simulation core.

26. Future Extensions [REVISED]

NEAT-style topology evolution is no longer a vague future line item — it is a fully specified Phase 7 deliverable (Appendix C). Remaining future extensions beyond the eight development phases:

- Vision based on rendered pixels instead of handcrafted sensor rays.
- Dynamic weather, day/night, and changing resources.
- Multi-agent cooperation and competition beyond predator/prey.
- Evolution of body parameters such as speed and energy efficiency.
- Parallelized population evaluation across multiple CPU cores or GPUs.
- 3D simulation using Godot or a dedicated simulator.
- Distributed experiment execution across machines.
- Behavior cloning or reinforcement-learning comparison baselines.

27. Expected Deliverables [REVISED]

Deliverable	Description
Source code	Modular Python project with tests, including batched-inference correctness tests
Simulation executable	Interactive artificial-life environment (GUI mode)
Configuration files	Reproducible experiment definitions, including reproducibility tier

Deliverable	Description
Experiment logs	Generation and agent metrics
Model checkpoints	Best genomes and network parameters
Analytics	Fitness and population visualizations
Statistical analysis report	Significance tests, confidence intervals, corrected comparisons (new)
Related work summary	Positioning against Sims, NEAT, and prior artificial-life platforms (new)
Technical report	Architecture, algorithms, methodology, and results
Demonstration	Live simulation showing evolutionary progress
Optional web dashboard	Browser-based monitoring and experiment visualization

28. Conclusion

This revision keeps v1.0's central design principle — the environment generates the problem, the neural network generates behavior, the fitness function evaluates consequences, and the genetic algorithm improves the population across generations — and closes the eleven gaps identified between that principle and its original specification. The system is now scoped honestly at every phase, batches population inference from the start rather than as an afterthought, calibrates its own fitness function before trusting it, and reports experimental results with the statistical rigor its “academic evaluation” framing calls for.

Final Design Baseline — v2.0

MVP (Phases 1–5): 2D world + food + obstacles + 100–250 agents + 7-ray sensors + a batched, calibrated small MLP + fitness + selection + crossover (compared across methods) + mutation + analytics with statistical comparison. Advanced version (Phases 6–7): reproduction + predators with hall-of-fame evaluation + a fully specified NEAT-style extension (Appendix C) + unseen-environment evaluation.

Appendix A. Recommended Initial Parameters [REVISED]

Parameter	Recommended baseline
World	1200 x 700
Population	250
Generation length	5000 simulation steps
Food / Obstacles	80 / 12
Sensor rays	7
Network	12-32-16-3
Selection	Tournament + 10% elitism
Crossover	Blend/arithmetic (default) — compared against uniform and mutation-only (Exp. E)

Parameter	Recommended baseline
Mutation rate / strength	0.05 / 0.10
Fitness normalization	Component scales calibrated against a random-policy and stand-still probe before weighting
Inference mode	Batched population forward pass (Sec. 12.3) — not per-agent module calls
Determinism settings	torch/numpy/random seeds fixed; deterministic algorithms enabled; device recorded as a reproducibility tier
Seeds per experiment	10 confirmatory (3 exploratory) — compared via Mann-Whitney U + bootstrap CI, Holm-Bonferroni corrected
Fast-forward	Up to 20x, headless mode only

Appendix B. Reference Pseudocode [REVISED]

```
def run_generation(population, world):
    for step in range(world.generation_steps):
        observations = batch_observe(population, world)          # (N, input_dim), Sec 12.3
        actions = batched_forward(observations, population.weights) # batched, not per-agent
        apply_actions(population, actions)
        world.update()                                           # fixed-timestep physics

    for agent in population:
        if agent.alive:
            agent.update_fitness(world)      # normalized components, Sec 14

    finalize_population(population)
    elites = select_elites(population)
    parents = select_parents(population)

    children = []
    while len(children) < len(population) - len(elites):
        a, b = choose_two(parents)
        genome = crossover(a.genome, b.genome, method=config.crossover_method) # Sec 13.2
        genome = mutate(genome)
        children.append(make_agent(genome))

    log_metrics(tag=config.reproducibility_tier)      # Sec 16.3
    return elites + children
```

Appendix C. NEAT-Style Extension Specification (Phase 7) [NEW]

This appendix is the resolution to C1: rather than a one-paragraph gesture at “species grouping,” Phase 7 has a concrete, literature-grounded specification to implement against.

C.1 Genome Representation

A Phase 7 genome is a list of node genes (input, hidden, output) and connection genes, where each connection gene carries a weight, an enabled/disabled flag, and an **innovation number**.

C.2 Historical Markings

A global innovation counter is shared across the population within a generation. When a structural mutation (add-connection or add-node) occurs, the system checks whether the same structural change (same connecting node pair) has already occurred elsewhere in the population this generation; if so, it reuses that innovation number, otherwise it assigns the next counter value. This is what makes crossover between differently-structured genomes meaningful — it is the direct structural fix to the competing-conventions problem named in C2.

C.3 Structural Mutation Operators

- **Add-connection:** a new weighted connection between two previously unconnected nodes.
- **Add-node:** an existing connection is disabled and split; a new node is inserted with two new connections, one initialized to weight 1.0 to minimize the immediate functional disruption.

C.4 Crossover for Variable-Topology Genomes

Genes are aligned by innovation number. Matching genes (present in both parents) are inherited randomly from either parent. Disjoint and excess genes (present in only one parent) are inherited from the fitter parent.

C.5 Compatibility Distance and Speciation

```
delta = (c1 * E) / N + (c2 * D) / N + c3 * W_avg_diff

# E = excess gene count, D = disjoint gene count
# N = gene count of the larger genome (N = 1 if both genomes are small)
# W_avg_diff = average weight difference over matching genes
# Recommended baseline: c1 = 1.0, c2 = 1.0, c3 = 0.4, species threshold = 3.0 (config)
```

Agents are grouped into species where delta is below the configured threshold. Fitness sharing divides each individual's fitness by its species' size, preventing one large species from dominating selection. Species that show no improvement for a configured number of generations are removed.

C.6 Complexification

Phase 7 populations start from a minimal topology — direct input-to-output connections only, no hidden nodes — and grow structure over generations via the mutation operators in C.3, rather than initializing from the fixed 12-32-16-3 architecture used in Phases 1–6.

C.7 Open Implementation Question — Batched Inference Under Variable Topology

Flag for early Phase 7 prototyping

The batched matmul approach in Section 12.3 assumes every agent shares one fixed architecture. That assumption breaks once genomes diverge structurally in Phase 7. Two candidate approaches — per-species batching (genomes within a species are structurally more similar) or a sparse/masked weight-matrix representation compatible with batched execution — should be prototyped early in Phase 7, not discovered late the way C3 was discovered late in v1.0.

Appendix D. Statistical Comparison Protocol [NEW]

The full procedure behind Section 20.5, resolving C11.

```

# For each experiment (A, B, NC, D, E):
final_fitness[condition] = [best_fitness_of_seed for seed in range(10)]    # per condition

for (cond_i, cond_j) in all_pairs(conditions):
    U_stat, p_raw = mann_whitney_u(final_fitness[cond_i], final_fitness[cond_j])
    ci_i = bootstrap_ci(final_fitness[cond_i], statistic=median, resamples=10000, alpha=0.05)
    ci_j = bootstrap_ci(final_fitness[cond_j], statistic=median, resamples=10000, alpha=0.05)
    effect_size = rank_biserial_correlation(final_fitness[cond_i], final_fitness[cond_j])
    record(cond_i, cond_j, U_stat, p_raw, ci_i, ci_j, effect_size)

p_corrected = holm_bonferroni(all_recorded_p_values)    # controls family-wise error rate
# A difference is reported as significant only if p_corrected < 0.05,
# and is always reported alongside its effect size, not p-value alone.

```

Appendix E. Proposed Project Title Variants [REVISED]

- Fixed-Topology Neuroevolution for Artificial Life, with a Fully Specified NEAT-Style Extension
- Design and Implementation of an Artificial Life Simulation Using Genetic-Algorithm-Optimized and NEAT-Evolved Neural Controllers
- Neuroevolution-Based Artificial Life: Fixed-Topology and Topology-Evolving Controllers — Design and Experimental Evaluation of Adaptive Agents